AI Assisted Coding Lab 15.4

SHIVANI PAMBIDI

BATCH-41

2303A52172

Question 1: Initial API Setup for a Service Platform

Prompt:

Generate a simple Flask application in Python to set up a basic API for a service platform. Create a root endpoint that returns a JSON response indica ng that the API is running (for example: {"status": "API is running"}). Ensure the server runs locally without errors and can be tested using a browser or an API testing tool like Postman. Include clear code and brief comments explaining how the API works and how to run it.

Code:

```python
from flask import Flask, jsonify

app = Flask(__name__)

@app.route('/status')
def status():
    return jsonify({"status": "API is live"})

if __name__ == "__main__":
    # In a Colab environment, 0.0.0.0 makes the server accessible within Colab's network.
    # 'debug=True' provides helpful debugging information.
    app.run(host='0.0.0.0', port=5000, debug=True)


import requests
import time

# Give the server a moment to start
time.sleep(3)

try:
    response = requests.get('http://127.0.0.1:5000/status')
    print(response.json())
except requests.exceptions.ConnectionError:
    print("Could not connect to the flask server. Make sure the flask app cell is running.")
```

Output:

```
 * Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
 * Running on all addresses (0.0.0.0)
 * Running on http://127.0.0.1:5000
 * Running on http://192.168.31.10:5000
Press CTRL+C to quit
 * Restarting with watchdog (windowsapi)
 * Debugger is active!
 * Debugger PIN: 178-865-502
```

Explanation:

This task helped me learn how a backend actually starts and runs in a real project. By creating a simple /status API, I understood how a server responds to requests and confirms that the system is working. I also learned how to run the Flask server and check the output, which is an important first step before building bigger features. Comple ng this task gave me a clear basic idea of how APIs work in real applications.

Question 2: Create API to Add and View Resources

Prompt:

Generate a Python Flask application that manages service requests stored in memory (using a list or dictionary). Create a GET endpoint /requests to return all service requests and a POST endpoint /requests to add a new request with fields like title and priority. Ensure that initially the GET request returns an empty list, and a er adding a new request using POST (for example: {" tle": "Internet issue", "priority": "High"}), it returns a confirmation along with the added data. Also include simple comments in the code and instructions on how to run and test the API.

Code:

```python
# !pip install Flask  # Uncomment and run this line first if Flask is not installed
import os
from flask import Flask, request, jsonify
app = Flask(__name__)
# In-memory storage for service requests
# Each request will be a dictionary with 'id', 'title', 'priority'
requests_db = []
next_request_id = 1
@app.route('/requests', methods=['GET'])
def get_requests():
    """Returns all service requests."""
    return jsonify(requests_db)
@app.route('/requests', methods=['POST'])
def add_request():
    """Adds a new service request."""
    global next_request_id
    data = request.get_json()
    if not data or not all(key in data for key in ['title', 'priority']):
        return jsonify({'error': 'Missing title or priority'}), 400
    new_request = {
        'id': next_request_id,
        'title': data['title'],
        'priority': data['priority']
    }
    requests_db.append(new_request)
    next_request_id += 1
    return jsonify({
        'message': 'Service request added successfully',
        'request': new_request
    }), 201
print("Starting Flask application...")
app.run(host='0.0.0.0', port=5000)
```

Output:

```
Starting Flask application...
 * Serving Flask app 'Lab_15'
 * Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
 * Running on all addresses (0.0.0.0)
 * Running on http://127.0.0.1:5000
 * Running on http://192.168.31.10:5000
Press CTRL+C to quit
```

Explanation:

The goal of this task was to create a simple web applica on using Python's Flask framework. This applica on allows us to manage service requests by providing two basic func onali es: f irst, we can retrieve a list of all exis ng service requests, and second, we can add new service requests, specifying their tle and priority. This setup demonstrates how to build a basic API that interacts with in-memory data, providing a founda on for understanding more complex web services.

Question 3: Modify Existing Resource Using AI Assistance

Prompt:

Generate a Python Flask application that extends an existing in-memory service request API. Use AI to generate a PUT endpoint /requests/ that updates a service request based on its index. The request should allow updating fields like title and priority. Ensure that if a valid index is provided, the API updates the request and returns a response like {"message": "Request updated", "request": {...}}. If an invalid index is given, handle it properly by returning a clear error message with HTTP status code 404. Include simple comments in the code and instructions on how to test the endpoint.

Code:

```python
import os
from flask import Flask, request, jsonify
app = Flask(__name__)
# In-memory storage for service requests
# Each request will be a dictionary with 'id', 'title', 'priority'
requests_db = []
next_request_id = 1
@app.route('/requests', methods=['GET'])
def get_requests():
    """Returns all service requests."""
    return jsonify(requests_db)
@app.route('/requests', methods=['POST'])
def add_request():
    """Adds a new service request."""
    global next_request_id
    data = request.get_json()
    if not data or not all(key in data for key in ['title', 'priority']):
        return jsonify({'error': 'Missing title or priority'}), 400
    new_request = {
        'id': next_request_id,
        'title': data['title'],
        'priority': data['priority']
    }
    requests_db.append(new_request)
    next_request_id += 1
    return jsonify({
        'message': 'Service request added successfully',
        'request': new_request
    }), 201
@app.route('/requests/<int:request_id>', methods=['PUT'])
def update_request(request_id):
    """Updates an existing service request by ID."""
    data = request.get_json()
    if not data:
        return jsonify({'error': 'No data provided for update'}), 400
    # Find the request by ID
    target_request = None
```

```python
    return jsonify({
        'message': 'Service request added successfully',
        'request': new_request
    }), 201
@app.route('/requests/<int:request_id>', methods=['PUT'])
def update_request(request_id):
    """Updates an existing service request by ID."""
    data = request.get_json()
    if not data:
        return jsonify({'error': 'No data provided for update'}), 400
    # Find the request by ID
    target_request = None
    target_index = -1
    for i, req in enumerate(requests_db):
        if req['id'] == request_id:
            target_request = req
            target_index = i
            break
    if target_request is None:
        return jsonify({'error': 'Service request not found'}), 404
    # Update fields if provided
    if 'title' in data:
        requests_db[target_index]['title'] = data['title']
    if 'priority' in data:
        requests_db[target_index]['priority'] = data['priority']
    return jsonify({
        'message': 'Service request updated successfully',
        'request': requests_db[target_index]
    }), 200
print("Starting Flask application...")
app.run(host='0.0.0.0', port=5000)
```

Output:

```
Starting Flask application...
 * Serving Flask app 'Lab_15'
 * Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
 * Running on all addresses (0.0.0.0)
 * Running on http://127.0.0.1:5000
 * Running on http://192.168.31.10:5000
Press CTRL+C to quit
```

Explanation:

The purpose of this extension was to enhance our simple service request applica on by adding the ability to modify exis ng requests. Previously, we could only view and create new requests. With the new 'PUT' endpoint, we can now update details like the tle or priority of a specific request, iden fied by its ID. This makes the applica on more dynamic and prac cal, as it allows for correc ons or changes to service requests a er they have been ini ally submi ed.

Question 4: Remove a Resource Safely

Prompt:

Generate a Python Flask application that adds a DELETE endpoint to an existing in-memory service request API. The endpoint `/requests/` should remove a service request based on its index. Ensure that when a valid index is provided, the API deletes the request and returns a response with a confirmation

message along with the deleted request details. If an invalid index is given, handle it properly by returning a clear error message without crashing the server. Include simple comments in the code and instructions on how to test the endpoint.

Code:

```python
import os
from flask import Flask, request, jsonify
app = Flask(__name__)
# In-memory storage for service requests
# Each request will be a dictionary with 'id', 'title', 'priority'
requests_db = []
next_request_id = 1
@app.route('/requests', methods=['GET'])
def get_requests():
    """Returns all service requests."""
    return jsonify(requests_db)
@app.route('/requests', methods=['POST'])
def add_request():
    """Adds a new service request."""
    global next_request_id
    data = request.get_json()
    if not data or not all(key in data for key in ['title', 'priority']):
        return jsonify({'error': 'Missing title or priority'}), 400
    new_request = {
        'id': next_request_id,
        'title': data['title'],
        'priority': data['priority']
    }
    requests_db.append(new_request)
    next_request_id += 1
    return jsonify({
        'message': 'Service request added successfully',
        'request': new_request
    }), 201
@app.route('/requests/<int:request_id>', methods=['PUT'])
def update_request(request_id):
    """Updates an existing service request by ID."""
    data = request.get_json()
    if not data:
        return jsonify({'error': 'No data provided for update'}), 400
    # Find the request by ID
    target_request = None
```

```python
def update_request(request_id):
        if req['id'] == request_id:
            target_request = req
            target_index = i
            break
    if target_request is None:
        return jsonify({'error': 'Service request not found'}), 404
    # Update fields if provided
    if 'title' in data:
        requests_db[target_index]['title'] = data['title']
    if 'priority' in data:
        requests_db[target_index]['priority'] = data['priority']
    return jsonify({
        'message': 'Service request updated successfully',
        'request': requests_db[target_index]
    }), 200
@app.route('/requests/<int:request_id>', methods=['DELETE'])
def delete_request(request_id):
    """Deletes a service request by ID."""
    global requests_db
    target_request = None
    target_index = -1
    for i, req in enumerate(requests_db):
        if req['id'] == request_id:
            target_request = req
            target_index = i
            break
    if target_request is None:
        return jsonify({'error': 'Service request not found'}), 404
    # Remove the request from the list
    deleted_request = requests_db.pop(target_index)
    return jsonify({
        'message': f'Service request with ID {request_id} deleted successfully',
        'deleted_request': deleted_request
    }), 200
print("Starting Flask application...")
app.run(host='0.0.0.0', port=5000)
```

Output:

```
Starting Flask application...
 * Serving Flask app 'Lab_15'
 * Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
 * Running on all addresses (0.0.0.0)
 * Running on http://127.0.0.1:5000
 * Running on http://192.168.31.10:5000
Press CTRL+C to quit
```

Explanation:

The purpose of this latest addition was to complete our service request management system by enabling the removal of requests. While we could previously view, create, and update requests, the absence of a delete function made the system incomplete. The new 'DELETE' endpoint allows users to permanently remove a specific service request using its unique ID, ensuring that the in-memory database accurately reflects only active and relevant requests. This capability is crucial for maintaining data hygiene and providing full CRUD (Create, Read, Update, Delete) functionality.

Question 5: AI-Assisted API Documentation and Readability

Prompt:

Generate a Python Flask application for a service request API and improve its readability using AI assistance. Add clear and meaningful docstrings and inline comments to all endpoints (GET, POST, PUT, DELETE), explaining the purpose, HTTP methods, input parameters, and response formats. Ensure that the code is easy for another developer to understand without reading the full implementation. Optionally, integrate automatic API documenta on using tools like Swagger or Flask-RESTX to display endpoint paths, methods, and sample request/response formats. Include instructions on how to run and access the documenta on UI.

Code:

```python
# !pip install flask flask-restx  # Install Flask and Flask-RESTx
import os
from flask import Flask, request, jsonify
from flask_restx import Api, Resource, fields  # Import Flask-RESTx components
app = Flask(__name__)
# Initialize Flask-RESTx API
api = Api(
    app,
    version='1.0',
    title='Service Request API',
    description='A simple API for managing service requests in memory',
    doc='/docs'  # Swagger UI available at /docs
)
# Create a namespace
ns = api.namespace('requests', description='Service Request operations')
# In-memory storage
requests_db = []
next_request_id = 1
# -------- Models for Swagger --------
request_input_model = ns.model('RequestInput', {
    'title': fields.String(required=True, description='The title of the service request'),
    'priority': fields.String(required=True, description='The priority of the service request')
})
request_output_model = ns.model('RequestOutput', {
    'id': fields.Integer(readonly=True, description='The unique identifier'),
    'title': fields.String(required=True, description='The title'),
    'priority': fields.String(required=True, description='The priority')
})
# -------- API Endpoints --------
@ns.route('/')
class RequestList(Resource):
    @ns.doc('list_requests')
    @ns.marshal_list_with(request_output_model)
    def get(self):
        """Get all service requests"""
        return requests_db
    @ns.doc('create_request')
```

```python
class RequestList(Resource):
    @ns.marshal_with(request_output_model, code=201)
    @ns.response(400, 'Validation Error')
    def post(self):
        """Create a new service request"""
        global next_request_id
        data = api.payload
        new_request = {
            'id': next_request_id,
            'title': data['title'],
            'priority': data['priority']
        }
        requests_db.append(new_request)
        next_request_id += 1
        return new_request, 201
@ns.route('/<int:request_id>')
@ns.response(404, 'Service request not found')
class Request(Resource):
    @ns.doc('update_request')
    @ns.expect(request_input_model, validate=True)
    @ns.marshal_with(request_output_model)
    def put(self, request_id):
        """Update a service request"""
        data = api.payload
        if not data:
            return {'message': 'No data provided for update'}, 400
        target_request = None
        target_index = -1
        for i, req in enumerate(requests_db):
            if req['id'] == request_id:
                target_request = req
                target_index = i
                break
        if target_request is None:
            return {'message': f'Service request with ID {request_id} not found'}, 404
        if 'title' in data:
            requests_db[target_index]['title'] = data['title']
```

```python
        if 'title' in data:
            requests_db[target_index]['title'] = data['title']
        if 'priority' in data:
            requests_db[target_index]['priority'] = data['priority']
        return requests_db[target_index], 200
    @ns.doc('delete_request')
    @ns.response(200, 'Service request deleted successfully')
    def delete(self, request_id):
        """Delete a service request"""
        global requests_db
        target_request = None
        target_index = -1
        for i, req in enumerate(requests_db):
            if req['id'] == request_id:
                target_request = req
                target_index = i
                break
        if target_request is None:
            return {'message': f'Service request with ID {request_id} not found'}, 404
        deleted_request = requests_db.pop(target_index)
        return {
            'message': f'Service request with ID {request_id} deleted successfully',
            'deleted_request': deleted_request
        }, 200
print("Starting Flask application...")
app.run(host='0.0.0.0', port=5000)
```

Output:

```
Starting Flask application...
 * Serving Flask app 'Lab_15'
 * Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
 * Running on all addresses (0.0.0.0)
 * Running on http://127.0.0.1:5000
 * Running on http://192.168.31.10:5000
Press CTRL+C to quit
```

Explanation:

This task helped me understand that writing code alone is not enough, it should also be easy for others to read and use. By adding docstrings and comments, I learned how to clearly explain what each API endpoint does, what inputs it takes, and what output it returns. This makes it easier for other developers to understand and work with the API without confusion. It also showed me the importance of proper documenta on in real projects, where multiple people work together.